Does "Hydro-ifying" a state machine make simulation cheaper? (negative result)

October 2026. Study across a toy, the MicroBus catchup client, and the Hydro Raft implementation.

The question

Customers adopt Hydro by keeping their service logic as a plain Rust state machine (`fn step(&mut State, Input)`) and hosting it inside one Hydro closure (`by_mut` in a `sliced!` block). Hydro does the plumbing and provides the simulator. The hope was that *decomposing* that opaque blob into Hydro-idiomatic operators (folds, `max`/`min`, keyed singletons) would make exhaustive simulation cheaper, and that this could be done one field at a time.

Answer: no. The simulator's cost does not come from the blob, and decomposition removes only a minor factor. Details below.

Where the simulator's cost comes from, hook by hook

The exhaustive simulator counts *decision sequences*. Decisions are made only at **hooks** that the sim builder inserts at specific points in the program. There is no state hashing and no dependency analysis: the simulator never looks inside a closure or at a struct. Every hook multiplies the number of executions.

Hook	Inserted when	Cost	Can a rewrite remove it?
Batch (`StreamHook`)	every `batch(...)` / `use::batch`	each tick run releases a non-empty subset of the items pending at the hook; the leftovers wait for a later tick. For n pending items on one stream at one member this is the number of ordered set partitions of n, F(n) = 1, 3, 13, 75, 541 for n = 1..5 (grows like n!). When several members have a runnable tick, the scheduler also forks on *which* runs next (m-way).	**No.** Which messages a tick sees is real timing.
Order (`StreamOrderHook`)	`assume_ordering` on a `NoOrder` stream; any `by_mut` closure on a `NoOrder` input; user `fold` or `reduce` on a `NoOrder` input	multiplies each batch of k items by k! (on top of the F(n) above)	**Yes**, by expressing the handler with the trusted builtins `max`, `min`, `count`, `sort`. `manual_proof!` does **not** remove it.
Network delivery	every intra-cluster channel	interleavings of deliveries	No.
Snapshot / membership / crash	`snapshot`, `with_dynamic_membership`, `with_crashable_*`	per option	No.

Only the Order hook is sensitive to program shape. Everything else is timing.

The intuition for why Hydro-ification should have helped

An opaque blob forces every input stream to be batched and fed, atomically, into one closure. The intuition was: "the simulator has to explore the product of every field's possible values and every intra-batch ordering, because it can't see which fields an output depends on." If the fields were separate Hydro collections — a `max` here, a keyed fold there — the simulator could see that most of them don't interact and skip those interleavings. Peeling one field at a time would then shrink the space incrementally.

What the evidence showed

The simulator does not explore fields; it explores hooks. Splitting a struct into ten singletons changes nothing unless it changes a hook. On a toy (3 unordered items, 1 tick): blob = 120 executions; same logic with `max`/`min`/`count` = 75; same logic with a hand-written `fold` or `reduce` = 120 again. The field layout was invisible; the one order hook set the number. Both numbers are the formulas above: 75 = F(4), the ordered set partitions of the 4 pending items into tick-batches; 120 is the same sum with every unordered block of size k weighted by k!.

Removing the order hook is all a rewrite can do, and it is worth about 2×.

Program / scenario	before	after (hook-free rewrite)	ratio
MicroBus burst test	670,680	498,480	1.35
MicroBus dedupe test	110,384	57,426	1.92
MicroBus (21 other tests)	1–5,904	same or ≤1.6×	~1
Raft, 3 members, election	10	8	1.25
Raft, 3 members, election + 2 heartbeats	1,000	512	1.95
Raft, 5 members, election	11,256	5,664	1.99
Raft, 3 members, election + commit 1 entry	51,480	31,232	1.65

The MicroBus numbers were checked exactly: summing 1/k! over the baseline's logged permutation decisions reproduces the "after" count to the integer. Nothing but the order hook moved.

The blow-up is batching, and no shape touches it. Raft grows ×8–10 per quiesced phase and ×25–45 per added member, before and after the rewrite. Those are the batch hooks on each member deciding which received messages enter which tick. That is why the Raft tests are fuzzed rather than exhaustive, and a port would not change it.

It is not incremental. The order hook belongs to the closure, not to a field. It disappears only when the closure's *last* opaque write over the unordered input is gone. Hoisting two of three fields buys zero. The unit is the handler (one per input stream), and each handler is a cliff.

The blobs were already well-behaved. MicroBus merged with `max`/`min` inside its closure; Raft sorted its message batch into canonical order inside `raft_step`. The ports did not add commutativity — they exposed it. Where a blob is genuinely order-dependent, the simulator's k! is correct and should stay.

Why the path shows little promise

1. The quantity people care about — simulation cost — is set by timing hooks that program shape cannot remove.
2. The one hook shape *can* remove is worth ~2×, and real tests already keep unordered bursts small (because k! made large ones unaffordable), so the factor stays small.
3. The win is per-handler and all-or-nothing, so there is no incremental story.
4. Proofs don't rescue the blob: machine-checked commutativity will attach to small lattice operators, not to a customer's `step` function, so any future simulator trust accrues only to code that was decomposed anyway.

What is left

- The hook model is exact and predictive. Reading a program's IR tells you its simulation cost before running it. That is a useful tool.

- The hook-free rewrites are free and strictly better (`sort()` before Raft's fold; `max`/ `min` for MicroBus's slot accounting). Worth landing, not worth pitching.
- The only lever that could change the *class* of cost is simulator-side: declining to fork on batch boundaries inside silent slices when the update is a proven monoid. For Raft that is a protocol-level invariance proof, not a lattice lemma.
- The case for Hydro-idiomatic state should therefore rest on types, provenance, and proof surface — not on simulation cost.